UGRC-I Report

Dynamic Louvain on GPUs with CUDA

Raadhes Chandaluru
Under Prof. Rupesh Nasre

November 11, 2025

Department of Computer Science and Engineering
Indian Institute of Technology Madras

Nov 2025

Abstract

This report details the design, implementation, and experimental evaluation of a dynamic Louvain algorithm tailored for high-performance community detection in graphs. Traditional static graph algorithms may not be sufficient for real-world networks that change over time. This project addresses this by modifying and developing a solution that efficiently maintains community structure under graph updates (edge additions/deletions).

To overcome the computational intensity of modularity optimization on massive datasets, the implementation leverages NVIDIA CUDA architecture for GPU acceleration. Key algorithmic phases, including modularity gain calculation and node movement, can be parallelized to maximize throughput and utilize the GPU's parallelism. We present an account of the CUDA kernels, data processing, optimization and synchronization strategies employed in the implementation.

Contents

1 Introduction **2**
- 1.1 Motivation . . . 2
- 1.2 Research Objectives . . . 2

2 Background **2**
- 2.1 The Louvain Community Detection Algorithm . . . 2
  - 2.1.1 Modularity . . . 3
  - 2.1.2 Algorithm . . . 4
  - 2.1.3 Phase 1: Modularity Optimization . . . 4
  - 2.1.4 Phase 2: Graph Aggregation . . . 5
- 2.2 CUDA and NVIDIA GPUs . . . 5

3 Related Work **6**
- 3.1 GPU-Based Static Louvain algorithm . . . 6
- 3.2 Dynamic Louvain algorithms . . . 7

4 Static Louvain Algorithm **7**

5 Work Theory **8**
- 5.1 General Considerations . . . 8
  - 5.1.1 Graph Representation . . . 8
    - 5.1.1.1 Adjacency Matrix . . . 8
    - 5.1.1.2 Compressed Sparse Row (CSR) Format . . . 8
  - 5.1.2 Community Node Tracking . . . 9
    - 5.1.2.1 Community Wise Storage . . . 9
    - 5.1.2.2 Vertex Wise Storage . . . 9
  - 5.1.3 Stopping Criterion . . . 10
  - 5.1.4 Calculating Modularity Gain . . . 10
- 5.2 Parallel Programming Considerations . . . 12
  - 5.2.1 Heuristics . . . 12
  - 5.2.2 Slight Change . . . 12
  - 5.2.3 Synchronization . . . 12
    - 5.2.3.1 Cooperative Kernels . . . 12
    - 5.2.3.2 Phase - 1 Synchronization . . . 13
  - 5.2.4 Graph Aggregation . . . 14
    - 5.2.4.1 Community Indicing . . . 14
    - 5.2.4.2 Modify Data Structures . . . 15
    - 5.2.4.3 Edge Aggregation . . . 15

6 Dynamic Louvain Algorithm **15**

7 Results **16**

8 Future Directions **17**

1 Introduction

1.1 Motivation

In today's interconnected world, nearly every complex system—from social media platforms and biological interactions to financial transactions and communication networks—can be modeled as a graph. A fundamental analytical task within these graphs is community detection, the process of partitioning a network into clusters (or communities) where nodes are densely connected internally but sparsely connected externally. Understanding this intrinsic grouping is crucial for tasks like influence maximization, targeted intervention in disease spread, and recommendation systems.

One of the most widely adopted and highly effective hierarchical algorithms for this task is the Louvain algorithm, renowned for its speed and ability to optimize the network's overall modularity. However, the Louvain algorithm was designed for static graphs, representing a single snapshot in time.

The majority of real-world networks are not static; they are dynamic, constantly evolving with the addition, deletion, and modification of nodes and edges. Applying a static algorithm like the classic Louvain method to a dynamic network requires rerunning the entire process from scratch every time an update occurs. This approach quickly becomes computationally prohibitive and introduces unacceptable latency when analyzing large-scale networks, rendering near real-time analysis impossible.

This project focuses on addressing this limitation by developing an efficient dynamic Louvain algorithm. The dynamic variant aims to quickly update the existing community structure in response to minor graph changes without needing to recalculate the modularity of the entire network.

1.2 Research Objectives

This project was originally based on implementing graph algorithms in starplat DSL NIBEDITA BEHERA and NASRE [2023]. Issues regarding compilation in certain constructs in starplat motivated us to focus on different graph algorithm problems.

- Understand the louvain community detection algorithm
- Research previous work done on optimizing this algorithm in the parallel setting and particularly on GPU architectures
- Expand upon the B.Tech Project work of Muthavarapu
- Implement the dynamic louvain algorithm in CUDA
- Introduce optimizations and benchmark the performance to judge the effectiveness of the various techniques used

2 Background

2.1 The Louvain Community Detection Algorithm

The louvain algorithm is a simple method to extract the community structure of large networks. It is a heuristic method that is based on modularity optimization. It is shown

to outperform all other known community detection method in terms of computation time. Moreover, the quality of the communities detected is very good, as measured by the so-called modularity. This is shown first by identifying language communities in a Belgian mobile phone network of 2.6 million customers and by analyzing a web graph of 118 million nodes and more than one billion links. The accuracy of our algorithm is also verified on ad-hoc modular networks. Blondel et al. [2008]

2.1.1 Modularity

Modularity is a fitness metric that is used to evaluate the quality of communities obtained by community detection algorithms (as they are heuristic based). It is calculated as the difference between the fraction of edges within communities and the expected fraction of edges if the edges were distributed randomly. It lies in the range [-0.5, 1] where higher is better. [U. Brandes and Wagner., 2007] The general formula for modularity in an weighted, undirected graph is defined as:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{1}$$

Where the elements are defined as follows:

- m: Represents the total weight of edges in the graph.
- i, j: Are indices representing any two nodes in the graph. The summation $\sum_{i,j}$ covers all possible pairs of nodes.
- A_{ij}: Is the weight of the edge between node i and j.
- k_i: Is the weighted degree of node i (sum of weights of edges incident to node i).
- $\delta(c_i, c_j)$: Is the Kronecker delta function. It is defined as:

$$\delta(c_i, c_j) = \begin{cases} 1 & \text{if node } i \text{ and node } j \text{ are in the same community } c \\ 0 & \text{otherwise} \end{cases}$$

It is often useful to consider the contribution of each community to the total modularity. The above formula for entire graph modularity can be simplified to the following expression for the modularity per community.

$$Q_c = \frac{\Sigma_{in}(c)}{2m} - \left(\frac{\Sigma_{deg}(c)}{2m} \right)^2 \tag{2}$$

Where the elements are defined as follows:

- Q_c is the modularity contribution of community c.
- $\Sigma_{in}(c) = \sum_{i \in c, j \in c} w_{ij}$ is the sum of the weights of all edges within community c with each edge being considered twice.
- $\Sigma_{deg}(c) = \sum_{i \in c} k_i$ is the sum of the degrees (weighted, if applicable) of all vertices belonging to community c.
- m is the total sum of the weights of all edges in the graph.

The first term, $\frac{E_{in}(c)}{2m}$, is the observed fraction of all edge weight that falls inside community c. The second term, $\left(\frac{\Sigma_{deg}(c)}{2m}\right)^2$, represents the expected fraction of edge weight inside community c if edges were placed randomly. Maximizing Q_c (and consequently the overall Q) means finding communities where the internal edge density significantly exceeds random expectation.

2.1.2 Algorithm

The louvain algorithm consists of two phases that are repeated until a stopping condition is reached. This can either be based on modularity or graph changes. The initial partition of the graph is each vertex being assigned to its own community.

2.1.3 Phase 1: Modularity Optimization

For each node i the modularity gain on moving it to each of its neighbours communities is evaluated. More precisely, the modularity gain of removing node i from the community it is currently assigned and moving into the community of each of its neighbours is evaluated. Node i is moved into the community where the maximum gain occurs where the gain is positive. This process is done for all nodes in the graph until moving any node to a neighbour community is not beneficial.

A critical observation regarding the stability and performance of the Louvain heuristic was provided by the original paper [Blondel et al., 2008] :

> Preliminary results on several test cases seem to indicate that the ordering of the nodes does not have a significant influence on the modularity that is obtained. However the ordering can influence the computation time.

Another important result is the efficient calculation of modularity gain:

$$\Delta Q_{i:a\to b} = \frac{1}{m}\left[(k_{i,b} - k_{i,a}) - \frac{k_i}{2m}\left(k_i + \sum_b - \sum_a\right)\right] \tag{3}$$

Where the new terms are defined as follows:

- $\Delta Q_{i:a\to b}$: The change in modularity resulting from moving node i from community a to community b.
- $k_{i,b}$: The sum of weights of edges between node i and nodes currently in community b.
- $k_{i,a}$: The sum of weights of edges between node i and nodes currently in community a.
- k_i: The total weighted degree of node i.
- $\sum_x$: The total sum of weighted degrees of nodes in community x before the movement.

2.1.4 Phase 2: Graph Aggregation

The communities that are formed via the first phase of the algorithm are merged into single nodes. All edges between community i and j are aggregated into a single edge with its weight being the sum of all weights between vertices that are were assigned to communities i and j respectively.

2.2 CUDA and NVIDIA GPUs

Knowledge of the CUDA Programming model, the computation hierarchy and GPU architecture is crucial is developing and optimizing algorithms based on CUDA for NVIDIA GPUs.

CUDA which stands for Compute Unified Device Architecture is NVIDIA's parallel computing platform and programming model that enables developers to utilize the GPUs. CUDA allows developers to code "kernels" a C/C++ function executed by threads simultaneously. These threads are organized into a hierarchical structure: a grid of blocks, where each block contains several warps of 32 threads each . This thread hierarchy maps directly onto the GPU hardware, where blocks are executed by the Streaming Multiprocessors (SMs). The CUDA model provides fine-grained control over memory management and synchronization of threads allowing developers to strategically place data in shared memory, or global memory to optimize data access and synchronization This allows performance tuning for domain specific applications and algorithms.

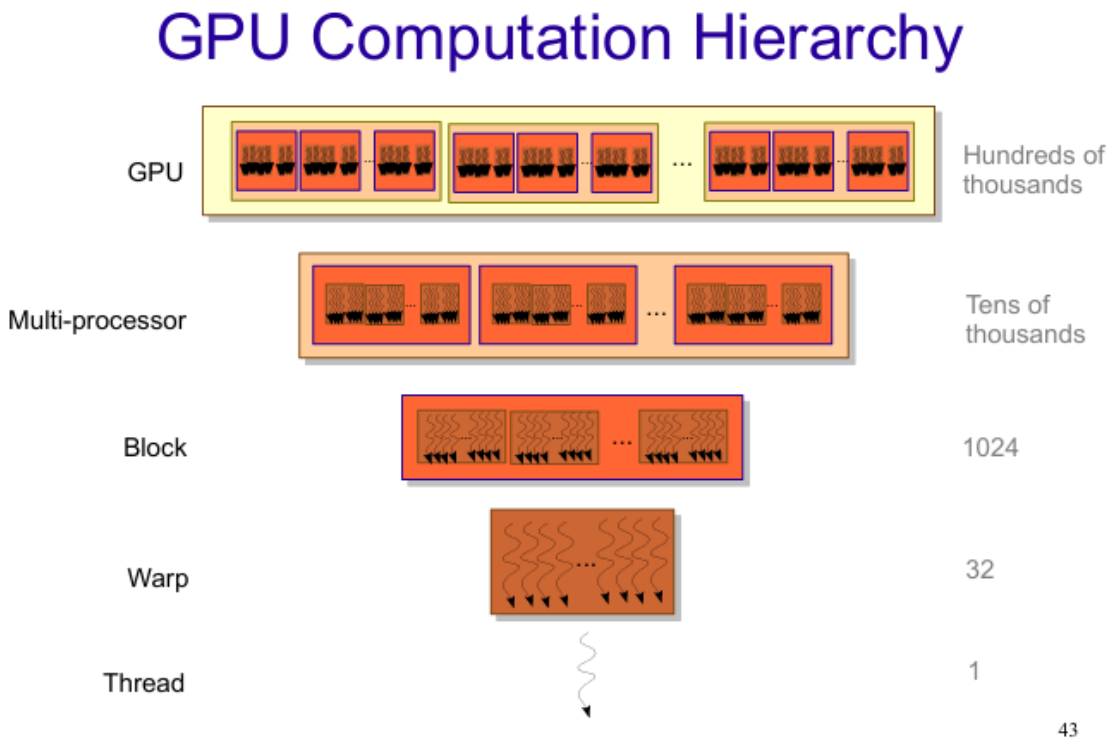

Figure 1: adapted from Nasre)

NVIDIA GPUs are fundamentally designed as parallel accelerators, which is different compared to traditional sequential CPUs. The core architectural feature is the Streaming

Multiprocessor (SM), a large collection of processing cores (CUDA Cores) optimized for parallel execution. A single modern GPU contains numerous SMs, each capable of running hundreds of threads concurrently. The RTX3090 GPU contains 82 SMs each with 128 CUDA cores. This design is built around high throughput rather than minimizing the latency of a single thread. Therefore GPUs are great in handling thousands of simultaneous operations. The memory hierarchy is crucial, including fast, on-chip shared memory accessible by threads within the same SM, along with large, high-bandwidth global memory. This structure makes GPUs ideal for highly parallel, data-intensive tasks like matrix operations.

3 Related Work

There is existing research regarding optimization on the the louvain algorithm as well as on developing efficient parallel implementations of the louvain algorithm for multi-core CPUs, GPUs, CPU-GPU hybrid systems and distributed systems.

3.1 GPU-Based Static Louvain algorithm

Bhowmik and Vadhiyar. [2019] The paper talks about a hybrid CPU-GPU approach with the core innovation being a heuristic to identify "doubtful vertices" that may be assigned to wrong communities due to partial graph information. These vertices are migrated in parallel between devices. There is an improvement of 42-73 % in execution times compared to state of the art CPU algorithms.

Anwesha Bhowmick The paper "Scalable multi-node multi-GPU Louvain community detection algorithm for heterogeneous architectures" by Anwesha Bhowmick, Sathish Vadhiyar, Varun PV. "In this work, we have developed heuristics to determine the target processors for migrating the doubtful vertices." They propose a multi-node and multi-device algorithm where independent computations on each node can be performed by GPU-only algorithm or hybrid GPU-CPU HyDetect.

Naim The paper "Community Detection on GPU" talks about parallelizing the access to individual edges. They fine tune the load balancing based on the degrees of nodes which can highly vary in large networks. The load balancing is achieved by scaling the number of threads per node according to its degree.

Maryam Mohammadi The paper "Accelerating Louvain community detection algorithm on graphic processing unit" proposes an adaptive CUDA Louvain method (ACLM) algorithm. Key innovations include dynamic resource management. Parallelization is done adaptively based on the available Streaming multiprocessors. They also efficiently use block level shared memory. In essence computation is optimized for the specific GPU hardware.

Han-Yi Chou The paper "Batched Graph Community Detection on GPUs" presents a "Nido" an efficient solution called batched vertex update "Compared to the coloring method, there is no preference order nor labeling of vertices. The vertices are divided into several user-defined batches and each batch is updated in a bulk-synchronous fashion". The paper talks about various libraries such as cuGraph, Galois, Gunrock and common heuristics including graph colouring although not used.

Nitin Gawande The paper "Towards scaling community detection on distributed-memory heterogeneous systems" introduces cuVite, which is a distributed multi-GPU C++ library for louvain community detection. They discuss irregularities for multi-GPU

systems and offer strategies to combat. They implement only the modularity optimization phase concurrently on GPU and CPU whereas the aggegration computation is done on the CPU. Their work exploits CUDA cooperative groups assigning one cooperative group per vertex and modifying the launch parameters based on vertex degree.

Chun Yew Cheong and Rick Siow Mong Goh The paper "Hierarchical Parallel Algorithm for Modularity-Based Community Detection Using GPUs" presents a hierarchical parallel algorithm with three levels. They first partitions the network into subnetworks, second they visit nodes in parallel, third the modularity computation of moving a node is done in parallel.

3.2 Dynamic Louvain algorithms

Barati The paper "C-Blondel: An Efficient Louvain-Based Dynamic Community Detection Algorithm" introduces a new louvain-based dynamic community detection algorithm. This aims to solve the community detection problem of social networks which iteratively change. Key ideas include "destructive nodes" and the idea of communities blowing up when such a destructive node is removed from the graph.

Some approaches for dynamic louvain community detection include the naive-dynamic algorithm presented in Thomas Aynaud and delta-screening approach in Neda Zarayeneh but this is not parallel. The naive-dynamic algorithm is simple and processes all vertices of the graph after any change in the graph. The delta-screening approach uses a modularity scoring mechanism to determine likely movements of vertices.

Sahu The paper "DF Louvain: Fast Incrementally Expanding Approach for Community Detection on Dynamic Graphs" builds upon Neda Zarayeneh translating the approach in a parallel algorithm. A dynamic frontier approach is also proposed with addresses overestimates of required processing in the naive-dynamic and delta-screening approaches.

Other work includes Cordeiro,

4 Static Louvain Algorithm

Below, the general pseudo code for the static louvain algorithm without implementation details. The basics have been explained in 2.1.2.

```
1    Intiial Graph G
2    Convert G to to community graph GC
3    Assign the community of each node to itself
4    while StoppingCriterion1 is False
5        while StoppingCriterion2 is False:
6            Iterate through edges, for each try to move
7            edge.src to the community of edge.dest if
8            there is modularity gain
9        Aggregate GC by combining nodes in the same community
10       into the same node. Aggregate edges between the same
11       two communities into one edge.
12   GC contains the final communities
```

5 Work Theory

There are quite a few steps to the louvain algorithm and many design choices must be made to code it up. In order to completely understand the intricacies of the implementation we start by implementating sequential versions of static louvain algorithm and dynamic louvain algorithms. Following this the parallel version of louvain algorithms can be implemented. As of this writing the the static louvain algorithm is implemented in the parallel setting, with a good idea of the framework for the dynamic louvain algorithm. As a result the following will focus more on the considerations for the static louvain algorithm.

5.1 General Considerations

5.1.1 Graph Representation

The first choice is the graph representation. It should ensure high performance in computation in parallel processing. The memory footprint and scalability should also be taken into consideration. The CSR representation is chosen for the CUDA-based algorithm whereas a simple adjacency list is chosen for the sequential algorithm.

5.1.1.1 Adjacency Matrix

The adjacency matrix is a simple representation for a graph $G = (V, E)$. Here V corresponds to the set of vertices and E corresponds to the set of edges of the graph. It is a matrix of dimension $n \times n$, where $n = |V|$. The value A_{ij} is the edge weight w_{ij} if an edge exists between node i and node j, and 0 otherwise.

- **Disadvantages:** Large real world networks (like social or citation graphs), are typically very sparse which leads to most entries begin zero. Storing this matrix requires $O(n^2)$ memory, which quickly becomes infeasible for graphs with millions of nodes.
- **Advantage:** It is useful for dense and small graphs. It is beneficial when quick constant-time lookup for any edge based on two endpoints is necessary.

5.1.1.2 Compressed Sparse Row (CSR) Format

The CSR format a common standard for storing large, sparse graphs and is the preferred input format for CUDA-based graph libraries (like cuGraph) and parallel algorithms. CSR does not store entries whose with edge weight zero making it memory efficient. The information can be stored in two arrays.

1. **Edge Information Array: (Data)** Stores the weights of the edges and the destination vertex node for the edge. This array can also be implemented in practice with two separate arrays. The edges for each starting vertex are grouped together and the groups of edges are stored in sorted order based on the starting vertex.
2. **Offset Information:** Stores the starting index for the adjacency list of each vertex in the `Data` array. It is notable that difference between consecutive entries $O[i + 1] - O[i]$ gives the degree of vertex i.

Memory Complexity: CSR requires $O(n + |E|)$ memory space. This is efficient for sparse graphs where $|E| \ll n^2$. The offsets array is powerful tool that can be used in finding the degree of vertices and efficiently parallelising algorithms.

5.1.2 Community Node Tracking

This might seem like a simple task; the simplest approach is just creating an array "community" that maps the vertex nodes to the community indices. However one needs to note that this will need to tracked across both the modularity optimization and aggregation phases.

5.1.2.1 Community Wise Storage

The first way would be to store the vertices in each community at each iteration of the algorithm.

```
vector<vector<int>> community_nodes(number of initial nodes);
Intialize community_nodes[i] to be a vector of just one element i
Do while stopping criterion is false
    // Process jth iteration
    // Phase 1
    // Phase 2
    Modify community_nodes based on the aggregation so that
    community_nodes[i] is now list of the initial vertices
    that are in the ith community after the jth iteration
```

5.1.2.2 Vertex Wise Storage

This method stores the community of each original vertex i at each iteration. The difference is the direction in which the information is stored. Previously it was community-centric, whereas this is vertex-centric.

```
// Initialization
community = new array[num_initial_nodes]
for i from 0 to num_initial_nodes - 1:
    community[i] = i
Do while stopping criterion is false
    // Process jth iteration
    // Phase 1
    // Phase 2
    Generate old_to_new_community_map, maps the (j-1)th
    iteration community to the jth iteration community
    in which it is moved to
    for i from 0 to num_initial_nodes - 1:
        old_community_indice = community[i]
        community[i] = old_to_new_community_map[
           old_community_indice]
```

Both these methods require $O(n)$ computation where n is the number of initial vertices, however the second method is more parallelizable. The community centric approach would require merging of several vectors which move to the same community. While this technically can be made efficient and parallelized if this were suppose a linked list that

would be a new method entirely. In fact, this would even be better as the computation would be proportional to the number of communities at the iteration. I stick to the vertex centric method for its simplicity in mapping.

5.1.3 Stopping Criterion

The louvain iterations require a stopping criterion. We additionally require a stopping criterion in louvain phase-1.

Move-Based: (No Modularity Gain) The first possible stopping criterion is when there is no possible move which increases the modularity of the graph. This is applicable in both.

Modularity Threshold: For phase-1's internal loop this can refer to the modularity increase over one pass of all graph edges begin less than a threshold. For louvain iterations this refers to the modularity increase due to one run of phase-1 and phase-2 being less than a threshold. Practically this is used in NetworkX Hagberg et al. [2008].

Loop Count: A limit on the total louvain iterations can be set. This will limit the communnity levels. Similarly a max-loop count can be set on the number of times the phase-1 loop passes over the edges of the graph. This can prevent infinite loops in parallel asynchronous implementations which may also be called thrashing. **Example:** Node A moves from one community to another, and in parallel Node Y moves as well. Due to each others moves, in the next loop iteration they both move back and so on.

Several libraries allow passing these parameters as arguments as threshold, tolerance, maxLevels and maxIterations. These libraries include AWS Neptune, NetworkX, Neo4j and Tigergraph.

5.1.4 Calculating Modularity Gain

Phase-1 of the louvain community detection algorithm involves moving nodes from one community to another based on whether this is modularity gain due to this. The calculation of modularity gain efficiently is an important problem to be solved here.

- **Naive:** Calculate modularity of the entire community arrangement before and after the node movement and subtract these values to obtain the modularity gain. In the most naive sense, calculating modularity would take $O(E + C)$ computational power where E is the number of edges and C is the number of communities at that point in time. 2.1.1

```
1  GetModularity()
2      community_in = new array[number of communitites]
3      community_deg = new array[number of communitites]
4  // Actually represents double the total due to edges being
5   repeated
6      total_edge_weight = 0
7      total_modularity = 0
8  // Each edge is listed twice (u,v) and (v,u) if not self-loop
9      for each edge in current aggregated graph network:
10         src_comm = community[edge.src]
11         if both endpoints of edge are within same community
12             community_in[src_comm] += edge.weight
13         community_deg[src_comm] += edge.weight
```

```
14          total_edge_weight += edge.weight
15      for each community_index from 0 to number of communities - 1:
16          in = community_in[community_index]
17          deg = community_deg[community_index]
18          total_modularity += in/total_edge_weight - (deg/
               total_edge_weight)^2
19      return total_modularity
```

- **Total Edge Weight:** Unnecessary to calculate each time but can stored as a global variable to avoid recomputation. In the dynamic algorithm the total edge weight should be changed whenever there are changes to the graph.
- **Community Parameters:** The community_deg and community_in can be pre-computed, modified when necessary and used during modularity calculation.
- Both of the above improvements will make modularity computation $O(C)$

Modularity gain can be calculated directly without calculating the total modularity of the graph.

- **Direct Gain Calculation:** It requires community_deg, node weighted degrees (k_i) and sum of weights of edges from a node i to initial and target (new) communities ($k_{i,a}$ and $k_{i,b}$). Given the community_deg is pre-computed as before, the computation boils down to finding k_i, $k_{i,a}$ and $k_{i,b}$. These can be computed with $O(E_i)$ where E_i refers to the number of edges incident to node i. 2.1.3

```
1  // total_edge_weight is again actually double due to
2  edge listing twice
3  GetModularityGain(move node i from initial_community to
      target_community)
4       k_i_old = 0
5       k_i_new = 0
6       k_i = 0
7       old_deg = community_degree[initial_community]
8       new_deg = community_degree[target_community];
9
10      for each edge incident to node i:
11          k_i += edge.weight;
12          if community[edge.dest] is initial_community
13              k_i_in_old += edge.weight
14          if community[edge.dest] is target_community
15              k_i_in_new += edge.weight
16
17      delta_modularity = 2.0 * (k_i_in_new - k_i_in_old) /
           total_edge_weight
18      delta_modularity += 2.0 * ((k_i * (old_deg - new_deg - k_i))
           / (total_edge_weight ^ 2));
19      return delta_modularity;
```

5.2 Parallel Programming Considerations

5.2.1 Heuristics

Graph Coloring heuristic is a common heuristic that comes up in parallel graph processing. The graph is colored so that adjacent vertices do not have the same color. The processing is done in parallel for vertices with the same color, and each color is processing sequentially. Such heuristics can be employed in a parallel louvain implementation as well, however the coloring step itself comes with overhead.

Asynchronous Updates is a heuristic that allows node movements to happen in parallel with graph structure being updated by multiple threads in parallel. The threads need to update shared arrays community and community_deg asynchronously. There can be instances when the graph is in an inconsistent state and a different thread makes decisions based on this.

- **Advantages:** Massive speed due to minimal locks, simplicity and the fact that adding more threads would result in a near linear speed up.
- **Disadvantages:** Threads can see inconsistent states and making decisions based on this. Thrashing can occur 5.1.3. There is more non determinism, higher variability in final results and is more than likely to have on average lower quality results.

5.2.2 Slight Change

The louvain algorithm indicates to move a node to the community with the best modularity gain. We slightly modify this for the our implementation to move a node if there is any modularity gain. While this is a deviation from the original algorithm it allows there is more simplicity. Disadvantage includes more work in critical sections and possibility of non-optimal movement of nodes.

5.2.3 Synchronization

5.2.3.1 Cooperative Kernels

In CUDA, developers write "kernels" which are special functions that run on the NVIDIA GPUs. CUDA provides support for parallelizing code in these kernels. Generally kernels are launched as:

```
1 // Kernel definition
2 __global__ void louvain_kernel(arguments) {
3     // processing
4 }
5
6 // Normal kernel launch
7 louvain_kernel<<<BLOCK_DIM, THREAD_DIM>>>(arguments)
```

A block is a abstraction of a group of threads. All threads in a block can be synchronized at a point in the kernel using "__syncthreads()". However synchronizing across blocks is not as simple. Developers would either need to implement this manually using atomics and loops or use cooperative kernels functionality (introduced in CUDA 9 toolkit). Through this there is CUDA support to synchronize all threads of all blocks that are launched within a cooperative kernel launch. This is demonstrated below:

```
// Kernel definition
__global__ void louvain_kernel(arguments) {
    cg::grid_group grid = cg::this_grid();
    // Processing
    grid.sync();
}

// Cooperative Kernel Launch
void* kernelArgs[] = {
        (void*)&arg1,
        (void*)&arg2,
        ...
    };
cudaLaunchCooperativeKernel((void*)louvain_kernel, BLOCK_DIM,
   THREAD_DIM, kernelArgs);
```

The **disadvantage** is having stricter limits on the launch. The GPU must run all blocks concurrently when cooperative kernels are used, which is the cause of stricter limits. We will use some cooperative kernels in the implementation for the ease of grid synchronization.

5.2.3.2 Phase - 1 Synchronization

One approach is asynchronous threads, although that would impede correctness. Correctness of the algorithm is crucial, hence we want to upload this, however speed is also needed. My approach includes manual locking mechanisms which uphold the correctness of the algorithm.

```
Thread index is tid, each tid is responsible for processing of
some edges in the graph
For each edge in the group of the thread tid
    moving_node = edge.src
    target_node = edge.dest
    if moving_node is in the same community
        as target_node continue

    lock on min(moving_node, target_node)
    lock on max(moving_node, target_node)

    initial_community = community[moving_node]
    target_community = community[target_node]

    recheck if initial_community is different from
    target_community, continue to next loop iteration
    if they are the same

    lock on min(initial_community, target_community)
    lock on max(initial_community, target_community)

    delta_modularity = GetModularityGain(move moving_node
        from initial_community to target_community)
```

```
24
25      if delta_modularity > eps
26          community[moving_node] = target_community
27          update degree of initial_community and target_community
28
29      unlock on max(initial_community, target_community)
30      unlock on min(initial_community, target_community)
31
32      unlock on max(moving_node, target_node)
33      unlock on min(moving_node, target_node)
```

GenModularityGain 5.1.4 uses the community_deg of the initial and target community. It also uses the community of the neighbours of the moving_node.

- The lock on the moving_node ensures that two threads cannot move the same node at the same time causing a race.
- The lock on the target_node ensures that any thread cannot move this node.
- The lock on the initial_community ensures no other node in the initial_community can be moved out and no node can be moved in at the same time.
- The lock on the target_community ensures no other node in the target_community can be moved out and no node can be moved in at the same time.

Why this upholds correctness: The movement of node A from community 1 to community 2 and movement of node B from community 3 to community 4 are allowed occur in parallel. Let us suppose the first is done by thread 1 and the second is done by thread 2. Without loss of generality let us show that the data write due to thread 1 does not affect thread 2. Thread 1 writes to community[A], community_degree[1] and community_degree[2]. Thread 2 reads community[B], community[x] where x is all of the neighbours of B, community_degree[3] and community_degree[4]. The only intersection occurs when node A is possibly a neighbour of node B so thread 2 accesses community[A] while thread 1 can write to it. However it is not important whether thread 2 reads the value before or after thread 1 writes due to the following explaination. The possible values of community[A] are 1 before the write by thread 1 or 2 after the write by thread 1. Regardless of which value that thread 2 sees, the behavior of GetModularityGain 5.1.4 is the same, as neither value is equal to the 3 nor 4 which are the initial_community and target_community respectively.

5.2.4 Graph Aggregation

Graph aggregation is an interesting step of the process. I have parallelized the graph aggregation in multiple steps. The main components of the community graph are the csr adjacency list, csr offsets, and community mapping from nodes to community during that iteration. There are other meta data including community_degree precomputation store, original node to community mapping.

5.2.4.1 Community Indicing

Count the number of formed communities and assign indices to each of the formed communities,

```
community_mapping = new array of zeroes
For each existing node, In Parallel:
    atomically set community_mapping[community[node]] to 1
sync the threads
community_count = 0
Iterate over each element x of community_mapping In Parallel
    Atomically add x to community_count
sync the threads
Find prefix sum of community_mapping
```

After this computation the community_count and 1-indexed indicing of communities is calculated. Suppose the index of the community that was formed was i, the new index is community_mapping[i] - 1. The prefix sum has been done with inclusive_scan from the cuda thrust library Nathan Bell.

5.2.4.2 Modify Data Structures

We need to modify the CSR adjacency list to reflect the new community indexes, recalculate community_degree, and modify the community array. For ease, I have the entire edge consisting of source, destination, weight in the csr adjacency list. It is basically a array of Edge Structs with an extra array of node offsets.

```
For each Edge in the CSR adjacency list In Parallel
    Edge.src = community_mapping[community[Edge.src]]
    Edge.dest = community_mapping[community[Edge.dest]]
Set community_degree array to 0
For each Edge in the CSR adjacency list In Parallel
    Atomically add Edge.weight to new_community_degree[Edge.src]
For i from 0 to community_count - 1 In Parallel
    community[i] = i
```

5.2.4.3 Edge Aggregation

Once the CSR adjacency list is updated to reflect the new community indices we can combined edges which correspond to same endpoints. CUDA thrust functions have been used heavily in this implementation Nathan Bell.

```
thrust sort edges in csr adjacency list based on the (src, dest)
    pair of edges, custom  edge comparator used here
thrust reduce by key invoked on the csr adjacency list with the
    keys being the (src, dest) pair of edges and the value to
    reduce being the edge weight
thrust for_each invoked on new csr adjacency list counting
    number of edges with each source. This is used to populate
    the csr offsets
```

6 Dynamic Louvain Algorithm

The typical idea is to first generate the communities from the initial graph using the static louvain implementation. Following this, edge additions, edge deletions are input to the algorithm online and processed.

The below is the initial dynamic louvain algorithm that has been implemented sequentially. There are problems in the below that need to be solved deeply.

```
Run static louvain code
Take Edge Inputs (Addition and Deletion) one at a time
    Create Queue of active_nodes, push the endpoints of the edge to the queue initially
    while active_nodes is Not Empty:
        Try to move the node at the front of the active_nodes queue to neighbouring
            communities if there is a modularity gain
        If there is a move, add the neighbours of the moved node to
            the active_nodes queue
```

Improvements to be done:

- Edge deletions can blow up communities, this is not handled currently above.
- We may need to add nodes from the initial and target community of the moved node.
- Parallelization of the algorithm.

7 Results

The GitHub repository for the codes and graphs tested can be found on GitHub.

Table 1: Experimental Environment and Software Versions

Component	**Specification**
Platform	Google Collaboratory
Hardware Accelerator	NVIDIA T4 GPU
GPU Driver Version	550.54.15
CUDA Version	12.4
C++ Compiler (Sequential)	g++ 11.4.0
Baseline Library	cugraph 25.08.00

- **graph3:** Karate Graph with 34 nodes and 76 edges.
- **nxgraph0:** Networkx erdos_renyi_graph with 100 nodes and 191 edges, with random weights between 1 and 10.
- **nxgraph1:** Networkx erdos_renyi_graph with 100 nodes and 194 edges, with random weights between 1 and 10.
- **nxgraph2:** Networkx erdos_renyi_graph with 300 nodes and 904 edges, with random weights between 1 and 10.
- **nxgraph3:** Networkx erdos_renyi_graph with 300 nodes and 879 edges, with random weights between 1 and 10.

Hagberg et al. [2008] Note that erdos_renyi_graph which is also known as binomial graph is just a random graph with each edge having an independent probability of occuring.

Table 2: Performance Comparison of Implementations (Executed on Google Colab)

Graph	Implementation	Time	Modularity	Comm.
graph3	GPU Static Louvain	9.0ms	0.309 051	9
	Sequential Static Louvain	18.0ms	0.295 229	11
	Cugraph GPU	14.1ms	0.415 600	9
nxgraph0	GPU Static Louvain	8.0ms	0.514 206	22
	Sequential Static Louvain	563.0ms	0.498 477	27
	Cugraph GPU	20.5ms	0.570 000	10
nxgraph1	GPU Static Louvain	9.0ms	0.520 144	18
	Sequential Static Louvain	460.0ms	0.499 594	26
	Cugraph GPU	19.9ms	0.555 300	14
nxgraph2	GPU Static Louvain	45.0ms	0.419 883	47
	Sequential Static Louvain	18 354.0ms	0.357 454	89
	Cugraph GPU	80.1ms	0.4684	27
nxgraph3	GPU Static Louvain	31.0ms	0.429 673	45
	Sequential Static Louvain	21 974.0ms	0.368 41	88
	Cugraph GPU	67.7ms	0.4798	25

Important Issue: There is an issue in the implementation where the graph is collapsed into a single community usually occurs when the input graph is dense sparse. The modularity will be 0 in this case. It is likely a synchronization error.

8 Future Directions

- Detailed profiling of time for different parts of code to identify bottlenecks
- Robustness tests on larger and varying types of graphs versus various existing implementations
- Trying various CUDA and GPU optimizations
- Implementing various heuristics and benchmarking to understand the effectiveness of such strategies
- Completing the implementation of dynamic louvain for GPUs

*Acknowledgments

I would like to acknowledge my guide Professor Rupesh Nasre for his valuable assistance during my project and Aditya Srivastava (CS22B066) for allowing the use of his report template.

I also acknowledge the use of AI-generative tools ChatGPT and Gemini for assistance in research and minimal portions of report (<15%) - For example conversion of my excel results to latex tabular format. Any AI-content has either been reviewed for consistency or processed by me and handwritten.

References

Varun PV Anwesha Bhowmick, Sathish Vadhiyar. Scalable multi-node multi-gpu louvain community detection algorithm for heterogeneous architectures. https://onlinelibrary.wiley.com/doi/full/10.1002/cpe.6987.

Mahsa Seifikar; Saeed Farzi; Masoud Barati. C-blondel: An efficient louvain-based dynamic community detection algorithm. https://ieeexplore.ieee.org/abstract/document/8982190.

A. Bhowmik and S. Vadhiyar. Hydetect: A hybrid cpu-gpu algorithm for community detection. https://cds.iisc.ac.in/faculty/vss/publications/anwesha-hydetect-hipc2019.pdf, 2019.

Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of statistical mechanics: theory and experiment*, 2008(10):P10008, 2008. doi: 10.1088/1742-5468/2008/10/P10008.

David Lo Chun Yew Cheong, Huynh Phung Huynh and 2013 Rick Siow Mong Goh. Hierarchical parallel algorithm for modularity-based community detection using gpus. https://link.springer.com/chapter/10.1007/978-3-642-40047-6_77.

Sarmento R.P. Gama Cordeiro, M. Dynamic community detection in evolving networks using locality modularity optimization. https://doi.org/10.1007/s13278-016-0325-1.

Aric Hagberg, Pieter Swart, and Daniel Chult. Exploring network structure, dynamics, and function using networkx. 06 2008. doi: 10.25080/TCWV9851.

Sayan Ghosh Han-Yi Chou. Batched graph community detection on gpus. https://dl.acm.org/doi/10.1145/3559009.3569655.

Mehdi Hosseinzadeh Maryam Mohammadi, Mahmood Fazlali. Accelerating louvain community detection algorithm on graphic processing unit. https://link.springer.com/article/10.1007/s11227-020-03510-9.

Harshitha Muthavarapu. Starplat dynamic louvain code. https://github.com/gajendra-iitm/starplat/tree/main/graphcode/dynamic_louvain_community_detection.

Fredrik; Halappanavar Mahantesh; Tumeo Antonino Naim, Md.; Manne. Community detection on the gpu. https://ieeexplore.ieee.org/abstract/document/7967153.

Rupesh Nasre. Professor rupesh nasre courses. https://cse.iitm.ac.in/~rupesh/teaching/.

Jared Hoberock Nathan Bell. Thrust: A productivity-oriented library for cuda. https://www.sciencedirect.com/science/chapter/edited-volume/abs/pii/B9780123859631000265.

Ananth Kalyanaraman Neda Zarayeneh. A fast and efficient incremental approach toward dynamic community detection. https://www.researchgate.net/publication/334523727_A_Fast_and_Efficient_Incremental_Approach_toward_Dynamic_Community_Detection.

EBENEZER RAJADURAI T SAI NITISH RAJESH PANDIAN M NIBEDITA BEHERA, ASHWINA KUMAR and RUPESH NASRE. Starplat: A versatile dsl for graph analytics. https://www.sciencedirect.com/science/article/pii/S074373152400131X, 2023.

Mahantesh Halappanavar Antonino Tumeo Ananth Kalyanaraman Nitin Gawande, Sayan Ghosh. Towards scaling community detection on distributed-memory heterogeneous systems. https://www.sciencedirect.com/science/article/pii/S0167819122000060.

Subhajit Sahu. Df louvain: Fast incrementally expanding approach for community detection on dynamic graphs. https://arxiv.org/pdf/2404.19634.

Jean-Loup Guillaume Thomas Aynaud. Static community detection algorithms for evolving networks. https://www.researchgate.net/publication/44270986_Static_community_detection_algorithms_for_evolving_networks.

M. Gaertler R. Gorke M. Hoefer Z. Nikoloski U. Brandes, D. Delling and D. Wagner. On modularity clustering. *IEEE transactions on knowledge and data engineering*, 2007.